

CUADERNO DE PRÁCTICAS // DESARROLLO WEB

UD1 • UD2 SPRING BOOT

De un proyecto vacío a una API REST

Kerin Aguilera

JAVA 21 • SPRING BOOT • OCTUBRE 2026

// Índice

- Primer intento: proyecto sin dependencias
- Segundo intento: una aplicación web
 - Configuración de Gradle
 - Rutas y parámetros
 - Página estática y formularios
 - Spring Initializr y empaquetado
 - Configuración del puerto
 - Objetos y respuestas JSON
 - Lectura del cuerpo de una petición
- Conclusión

// 1. Primer intento: proyecto sin dependencias

La práctica comienza creando un proyecto Spring Boot sin dependencias adicionales. En Spring Initializr, el único cambio respecto a la configuración inicial es seleccionar Java 21.

The screenshot shows the 'New Module' dialog in Spring Initializr. The left sidebar lists 'Generators' with 'Spring Boot' selected. The main form contains the following fields:

- Server URL: start.spring.io
- Name: proyecto-UD1-UD2
- Location: ~/Escritorio/CARPETA DE CLASES/DWS/PROYECTOS
- Language: Java (selected), Kotlin, Groovy
- Type: Gradle - Groovy (selected), Gradle - Kotlin, Maven
- Group: Ruvik
- Artifact: proyecto-UD1-UD2
- Package name: ruvik.proyectoud1ud2
- JDK: Project JDK 21
- Java: 21
- Packaging: Jar (selected), War
- Configuration: Properties (selected), YAML

At the bottom right, there are 'Next' and 'Cancel' buttons.

En el siguiente paso dejamos vacía la lista de dependencias.

GraalVM Native Support

Support for compiling Spring applications to native executables using the GraalVM native-image compiler.

Added dependencies:

No dependencies added

Previous

Create

Cancel

El proyecto generado incluye una clase de entrada sencilla:

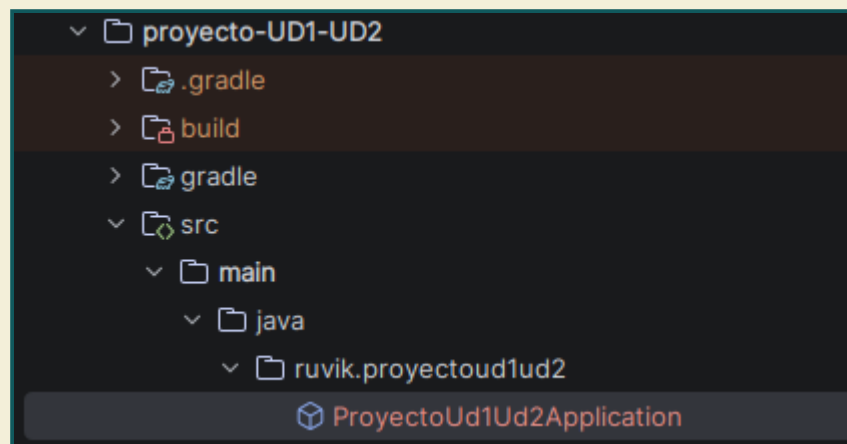
```
package ruvik.projectoud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class ProyectoUd1Ud2Application {

    public static void main(String[] args) {
        SpringApplication.run(ProyectoUd1Ud2Application.class, args);
    }
}
```

La estructura inicial queda así:



Al ejecutar esta aplicación, el proceso termina poco después de iniciarse. Sin una dependencia web, Spring Boot no levanta un servidor HTTP que mantenga la aplicación escuchando peticiones.

```
8:58:47: Executing 'ruvik.projectoud2.ProyectoUd1Ud2Application.main()'.
> Task :compileJava
> Task :processResources
> Task :classes

> Task :ruvik.projectoud2.ProyectoUd1Ud2Application.main()
...
BUILD SUCCESSFUL in 2s
3 actionable tasks: 3 executed
Consider enabling configuration cache to speed up this build: https://docs.gradle.org/7.1/userguide/configuration_cache_enabled.html
8:58:49: Execution finished 'ruvik.projectoud2.ProyectoUd1Ud2Application.main()'.
```

Por eso, al intentar utilizar anotaciones web del **inicio rápido de Spring**, faltan las clases necesarias para compilar. La aplicación base arranca, pero todavía no puede atender peticiones web.

```
ProjectoUd1Ud2Application.java x
1 package ruvik.projectcloud1ud2;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 @SpringBootApplication
7 @RestController
8 public class ProyectoUd1Ud2Application {
9
10     public static void main(String[] args) {
11         SpringApplication.run(ProyectoUd1Ud2Application.class, args);
12     }
13
14     @GetMapping("/hello") no usages
15     public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
16         return String.format("Hello %s!", name);
17     }
18
19 }
20
```

AVISO DE RUTA //

La primera prueba deja clara la diferencia entre crear una aplicación Spring Boot y añadirle las capacidades necesarias para convertirla en una aplicación web.

// 2. Segundo intento: una aplicación web

Creamos otro proyecto con la misma configuración y añadimos la dependencia **Spring Web**. Esta incorpora lo necesario para crear controladores y atender peticiones HTTP.

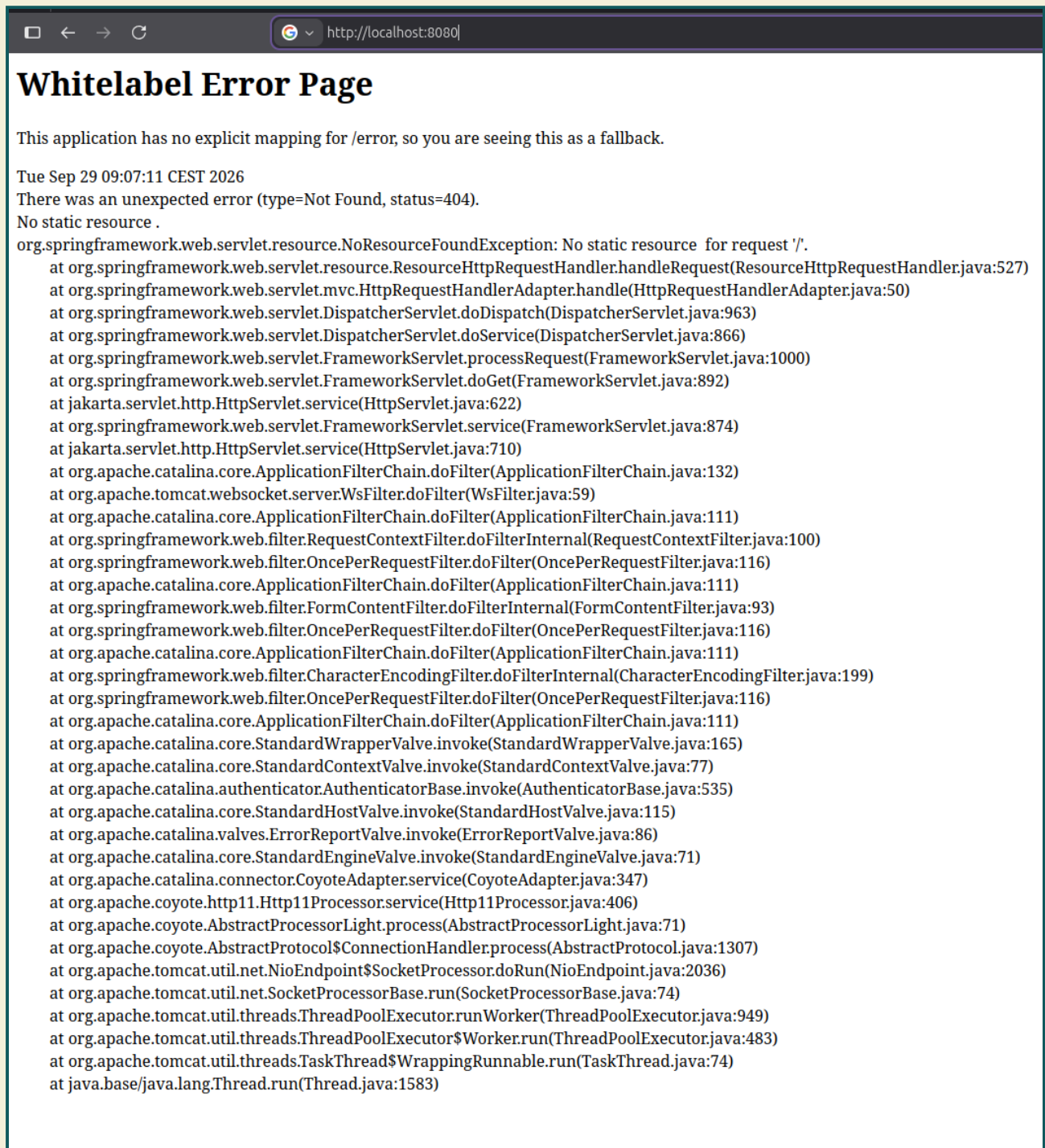
```
Ud1Ud2Application.java x
1 package ruvik.ud1ud2;
2
3 import org.springframework.boot.SpringApplication;
4 import org.springframework.boot.autoconfigure.SpringBootApplication;
5
6 import org.springframework.web.bind.annotation.GetMapping;
7 import org.springframework.web.bind.annotation.RequestParam;
8 import org.springframework.web.bind.annotation.RestController;
9
10
11 @SpringBootApplication
12 @RestController
13 public class Ud1Ud2Application {
14
15     public static void main(String[] args) {
16         SpringApplication.run(Ud1Ud2Application.class, args);
17     }
18
19     @GetMapping("/hello")
20     public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
21         return String.format("Hello, %s!", name);
22     }
23
24 }
25
```

Tomcat en el puerto 8080:

```
:: Spring Boot :: (v4.1.1)

Tomcat initialized with port 8080 (http)
Starting service [Tomcat]
```

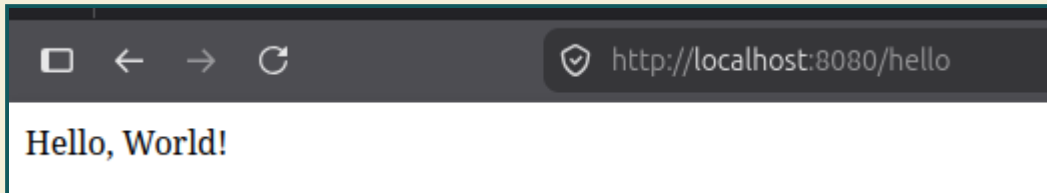
Al visitar `http://localhost:8080/` aparece un error porque todavía no hemos definido una ruta para la raíz.



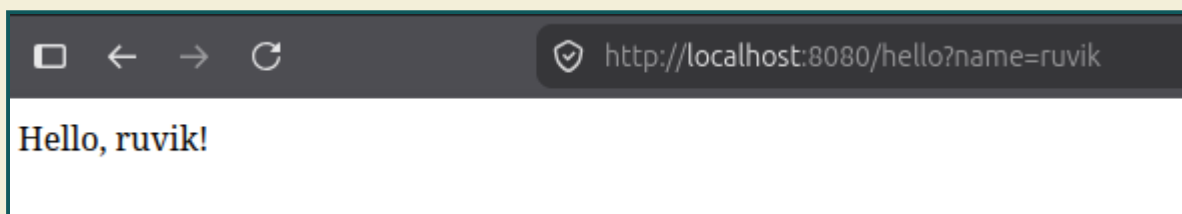
El controlador de ejemplo responde en `/hello`:

```
@GetMapping("/hello")
public String hello(@RequestParam(value = "name", defaultValue = "World") String name) {
    return String.format("Hello, %s!", name);
}
```

La ruta `http://localhost:8080/hello` devuelve el saludo por defecto.



El parámetro `name` permite personalizarlo: `http://localhost:8080/hello?name=ruvik`.



Configuración de Gradle

El archivo de Gradle declara los plugins, la versión de Java y las dependencias del proyecto.

```
1  plugins {
2      id 'java'
3      id 'org.springframework.boot' version '4.1.1'
4      id 'io.spring.dependency-management' version '1.1.7'
5  }
6
7  group = 'Ruvik'
8  version = '0.0.1-SNAPSHOT'
9  description = 'UD1-UD2'
10
11 java {
12     toolchain { JavaToolchainSpec it ->
13         languageVersion = JavaLanguageVersion.of(21)
14     }
15 }
16
17 repositories {
18     mavenCentral()
19 }
20
21 dependencies {
22     implementation 'org.springframework.boot:spring-boot-starter-webmvc'
23     developmentOnly 'org.springframework.boot:spring-boot-devtools'
24     testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
25     testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
26 }
27
28 tasks.named('test') { Task it ->
29     useJUnitPlatform()
30 }
31
```

Los plugins identifican Java, Spring Boot y la gestión de dependencias:

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.1'
    id 'io.spring.dependency-management' version '1.1.7'
}
```

La configuración del proyecto especifica Java 21 y Maven Central como repositorio:

```
group = 'Ruvik'
version = '0.0.1-SNAPSHOT'
description = 'UD1-UD2'

java {
    toolchain {
        languageVersion = JavaLanguageVersion.of(21)
    }
}

repositories {
    mavenCentral()
}
```

Las dependencias principales son Spring MVC, DevTools y las herramientas de prueba:

```
dependencies {
    implementation 'org.springframework.boot:spring-boot-starter-webmvc'
    developmentOnly 'org.springframework.boot:spring-boot-devtools'
    testImplementation 'org.springframework.boot:spring-boot-starter-webmvc-test'
    testRuntimeOnly 'org.junit.platform:junit-platform-launcher'
}
```

`implementation` incorpora dependencias necesarias en la aplicación; `developmentOnly` se reserva para el desarrollo, y `testImplementation` para las pruebas.

AVISO DE RUTA //

Esta es mi interpretación de la configuración que estoy viendo. La finalidad de cada dependencia puede variar según el proyecto.

Rutas y parámetros

También podemos traducir la ruta y el parámetro al español. Con `@RestController`, el valor devuelto por el método se escribe directamente en la respuesta HTTP.

```
package ruvik.ud1ud2;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

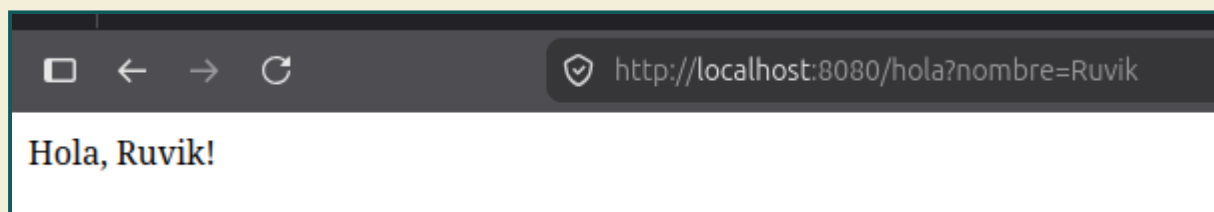
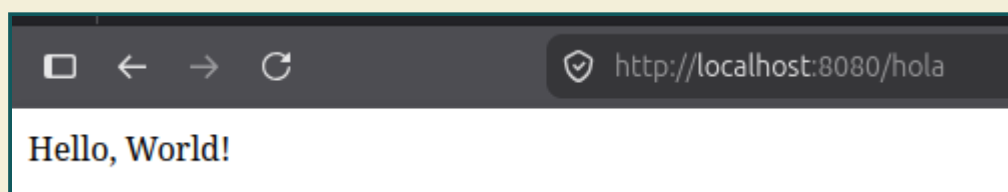
@SpringBootApplication
@RestController
public class Ud1Ud2Application {

    public static void main(String[] args) {
        SpringApplication.run(Ud1Ud2Application.class, args);
    }

    @GetMapping("/hola")
    public String hello(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
        return String.format("Hola, %s!", nombre);
    }
}
```

`http://localhost:8080/hola` utiliza el valor predeterminado;

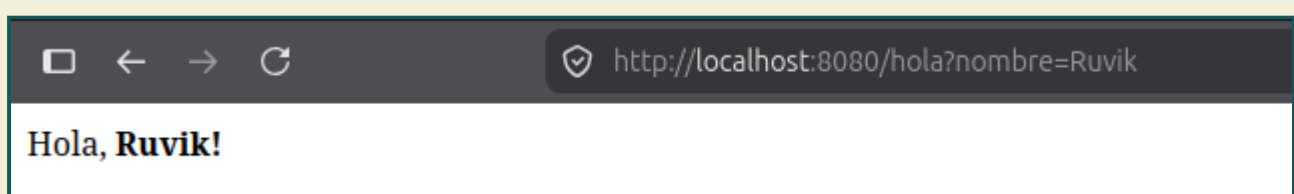
`http://localhost:8080/hola?nombre=ruvik` utiliza el que enviamos en la URL.

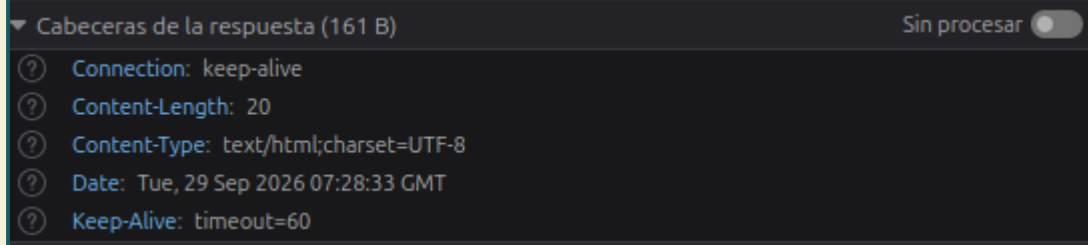


Durante la prueba también se incluyó HTML en la cadena de respuesta:

```
return String.format("Hola, <b>%s!</b>", nombre);
```

El navegador muestra el saludo en negrita. El formato concreto de la respuesta depende del controlador y del tipo de contenido enviado.





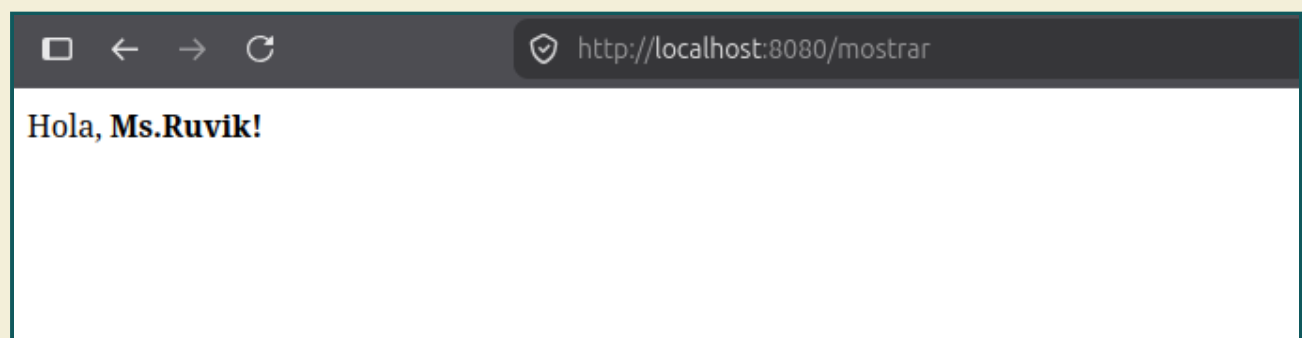
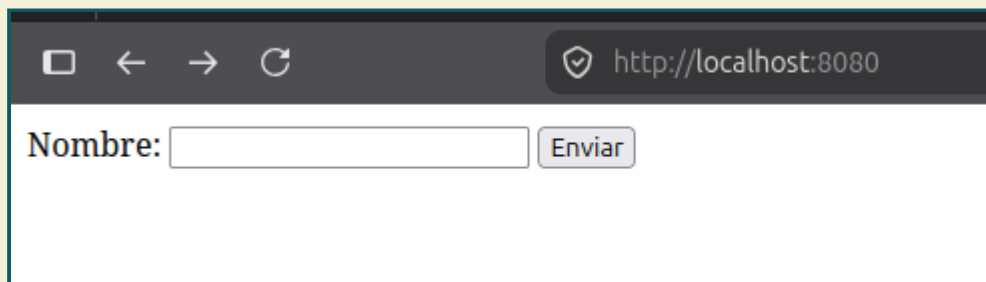
Página estática y formularios

Una página llamada `index.html`, ubicada en los recursos estáticos de la aplicación, se puede servir en `/`. Este formulario envía el nombre al servidor mediante una petición POST:

```
<html>
  <body>
    <form action="/mostrar" method="post">
      <label for="nombre">Nombre:</label>
      <input id="nombre" type="text" name="nombre">
      <button type="submit">Enviar</button>
    </form>
  </body>
</html>
```

El controlador recibe el parámetro y construye la respuesta:

```
@PostMapping("/mostrar")
public String mostrar(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
    return String.format("Hola, <b>%s!</b>", nombre);
}
```



La diferencia entre `@Controller` y `@RestController` importa al devolver una cadena. En un `@Controller`, Spring interpreta normalmente la cadena como el nombre de una vista.

`@ResponseBody` indica que debe escribirse en el cuerpo de la respuesta;

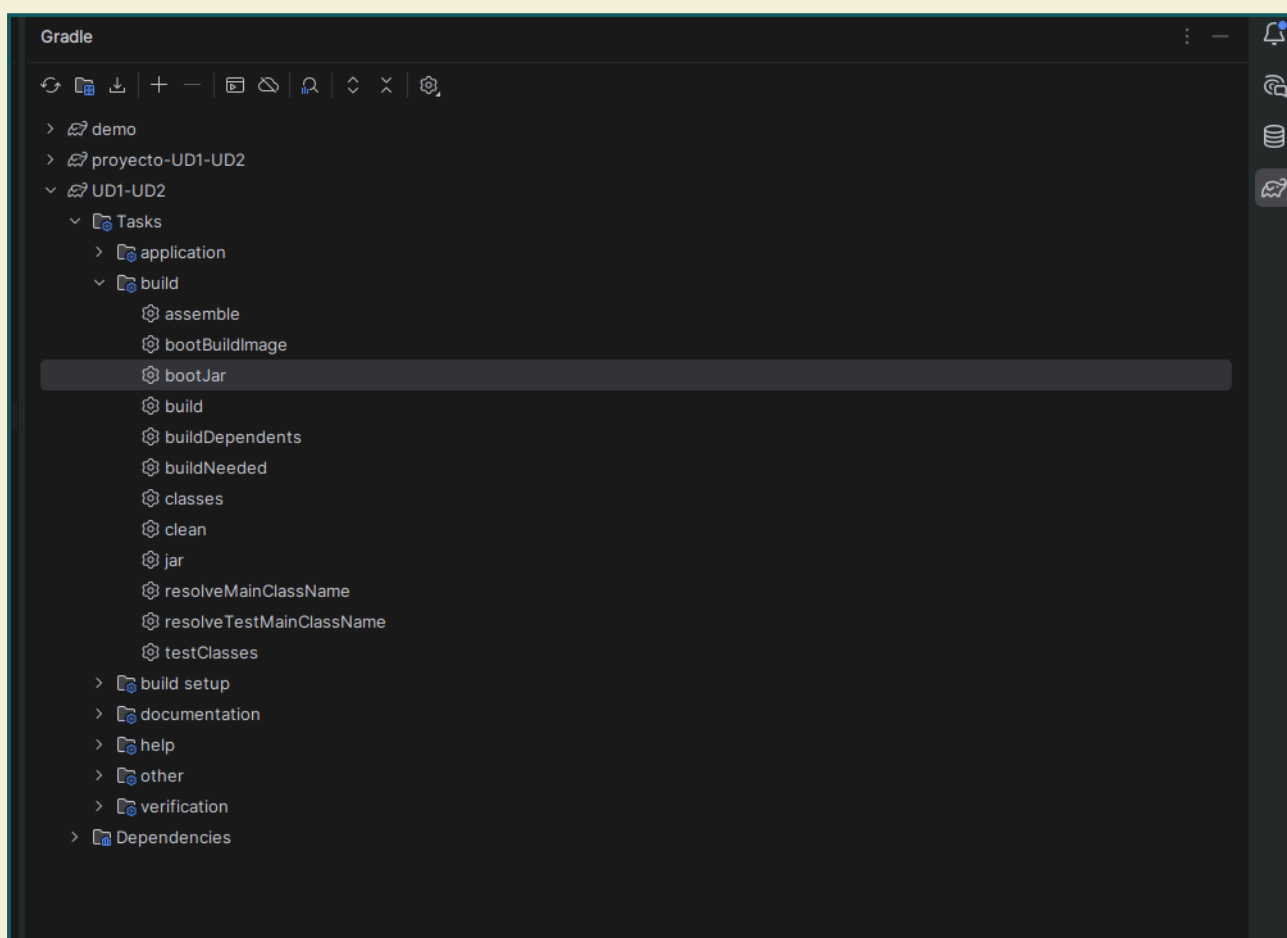
`@RestController` aplica ese comportamiento^{11/15} a todos los métodos del controlador.

```
@PostMapping("/mostrar")
@ResponseBody
public String mostrar(@RequestParam(value = "nombre", defaultValue = "Mundo") String nombre) {
    return String.format("Hola, <b>%s!</b>", nombre);
}
```

Spring Initializr y empaquetado

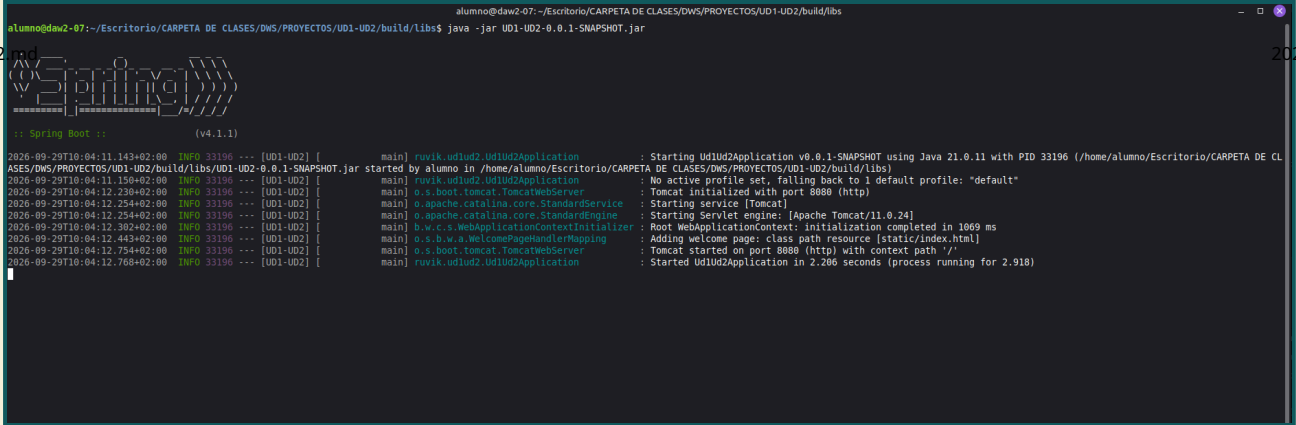
Spring Initializr simplifica la creación del proyecto: permite seleccionar lenguaje, versión y dependencias, y genera una estructura inicial lista para desarrollar.

En IntelliJ, la tarea **BootJar** genera el archivo ejecutable:



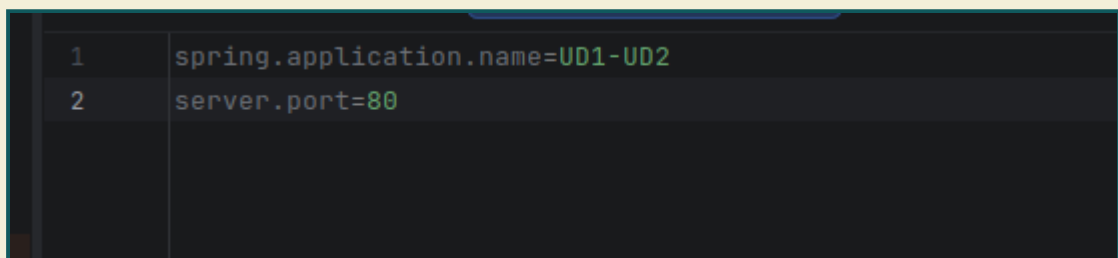
En el panel de Gradle, la ruta es **Tasks > build > BootJar**. El archivo generado queda en **build/libs**. Se puede ejecutar desde una terminal así:

```
java -jar build/libs/nombre-del-proyecto.jar
```

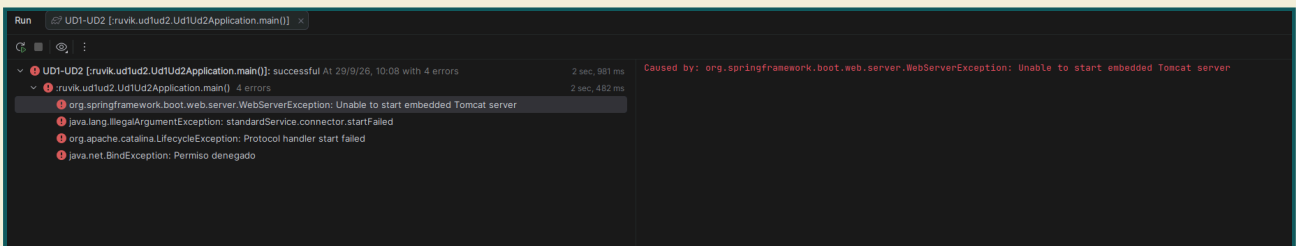


Configuración del puerto

El puerto del servidor se puede cambiar en `application.properties`, por ejemplo con `server.port=80`.



En Linux y otros sistemas tipo Unix, los puertos inferiores a 1024 suelen requerir permisos especiales. Por eso, una aplicación ejecutada como usuario normal puede no tener permiso para enlazarse al puerto 80. Para desarrollo local conviene usar un puerto no privilegiado, como el 8080.

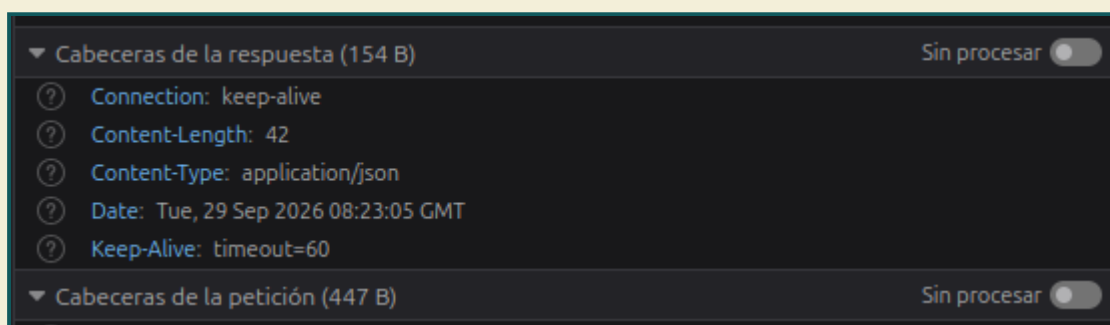
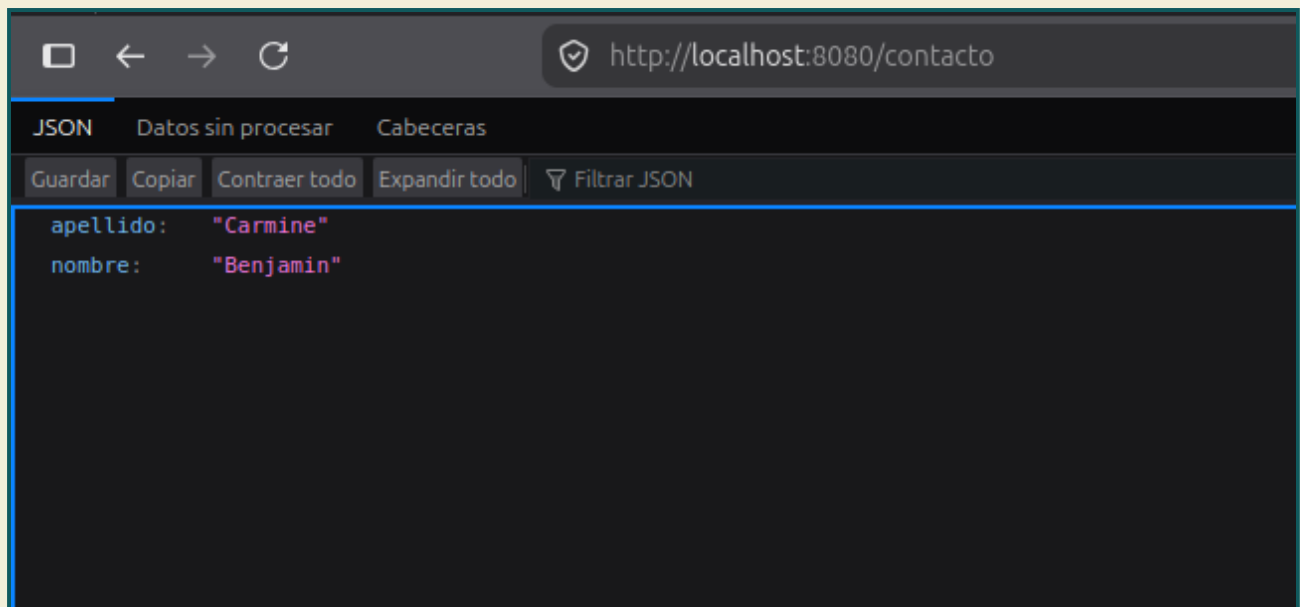


Objetos y respuestas JSON

Al devolver un objeto Java desde un controlador REST, Spring lo serializa como JSON si hay un conversor compatible disponible en el proyecto. En este ejemplo, se crea un `Contacto` a partir de dos parámetros:

```
@GetMapping("/contacto")
public Contacto contacto(
    @RequestParam(value = "nombre", defaultValue = "Benjamin") String nombre,
    @RequestParam(value = "apellido", defaultValue = "Carmine") String apellido) {

    Contacto contacto = new Contacto();
    contacto.setNombre(nombre);
    contacto.setApellido(apellido);
    return contacto;
}
```



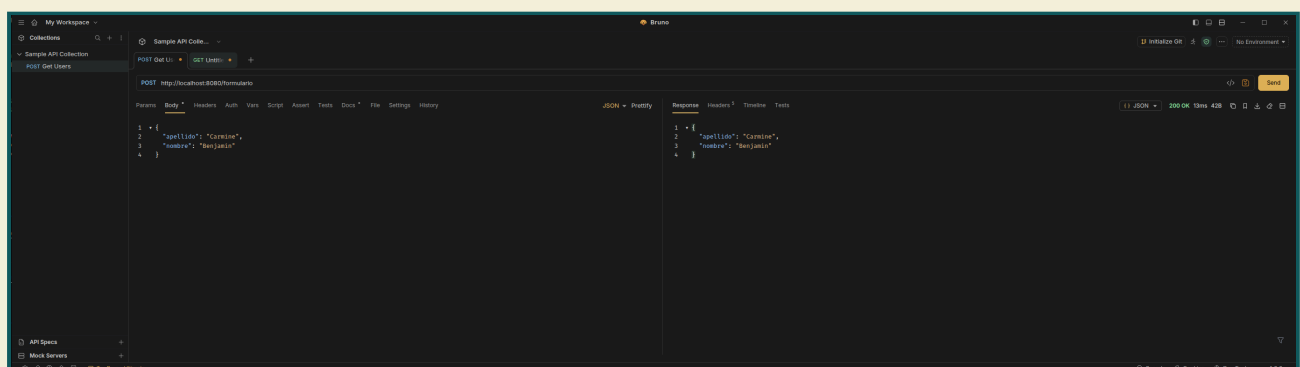
Lectura del cuerpo de una petición

Para recibir un objeto enviado en el cuerpo de una petición POST, se puede usar

`@RequestBody`. El ejemplo devuelve el objeto recibido:

```
@PostMapping("/formulario")
public Contacto formulario(@RequestBody Contacto contacto) {
    return contacto;
}
```

En Bruno, se envía un cuerpo JSON que Spring convierte en un objeto `Contacto`:



Response Headers 5 Timeline Tests	
200 OK 13ms 42B	
Name	Value
content-type	application/json
content-length	42
date	Tue, 29 Sep 2026 08:38:27 GMT
keep-alive	timeout=60
connection	keep-alive

`@RestController` y `@ResponseBody` permiten escribir directamente el resultado en la respuesta. Si se necesita controlar también el estado HTTP o las cabeceras, `ResponseEntity` ofrece esas opciones.

// 3. Conclusión

Volver a trabajar estos conceptos me permitió avanzar con más rapidez: ya entendía mejor por qué se necesitan ciertas dependencias y cómo se conectan las rutas con las respuestas. Todavía me sentía algo oxidado con los POJOs, pero pude completar las pruebas con más soltura que la primera vez.

La idea principal que me llevo es que Spring Boot facilita el arranque, pero las dependencias y las anotaciones elegidas determinan cómo se comporta la aplicación: si mantiene un servidor web activo, cómo recibe datos y cómo construye cada respuesta.